

A Generator–Realizer Architecture for Constrained Predictive Physical Human–Robot Interaction

Draft status. This document is a new research direction, not a revised presentation of the existing double-integrator MPC paper. The numerical claims in Sections 7 and 8 are generated by the code in this folder: Section 7 is a planar point-mass proof of concept, and Section 8 is the same generator–realizer architecture instantiated on a torque-controlled 7-DOF Franka FR3 in MuJoCo. The FR3 study is a focused core (impedance and admittance generators, horizon-wide torque feasibility, slack-relaxed workspace/speed bounds, a reactive-clipping comparator, one benchmark scenario) rather than the full validation program a manipulator-level claim would eventually need; human-participant, hardware, and comparisons against the broader predictive-impedance literature remain future work (Section 8.4).

Abstract

Physical human–robot interaction controllers typically couple two decisions that are conceptually separate: what interaction behavior is desired, and how a constrained robot delivers it. This paper separates them architecturally. A pluggable interaction-dynamics generator specifies a desired contact-port acceleration $a_k^{\text{id}} = f_\theta(e_k, \dot{e}_k, F_{h,k})$; a single receding-horizon realization layer tracks that behavioral acceleration — not a motion trajectory — under actuator, command-rate, speed, and workspace constraints, and exposes the gap between what was requested and what the constrained robot delivered as an explicit realization residual. The realization layer's cost, constraints, and decision variable are fixed once, independent of which generator is plugged in: we state and prove this generator-independence property, then validate it on two affine instances, linear impedance and force-guided admittance, realized by the identical QP without redesign.

In a reproducible planar simulation, the same predictive layer is used without retuning for both generators. When the unconstrained reference behavior would leave a 0.10 m workspace, instantaneous realization followed by command clipping reaches 0.266 m for the impedance generator and 0.525 m for the admittance generator. Predictive realization limits displacement to 0.100002 m and 0.0912 m, respectively, while respecting the 0.22 m/s component-wise speed bound to numerical tolerance. The required deviation from the desired dynamics is exposed as a measurable realization residual rather than hidden by parameter adaptation. A further experiment reassigns the

generator on a single running controller instance online — impedance, then admittance, then impedance again, mid-simulation, under a sustained disturbance — and shows no discontinuity beyond what the controller's own command-rate limit already permits, the direct experimental counterpart to the independence theorem.

We then instantiate the same architecture on a torque-controlled 7-DOF Franka FR3 in MuJoCo, freezing the task-space inertia, feedforward torque, and Jacobian at each solve (a local approximation, not exact horizon-wide prediction) and enforcing per-joint torque feasibility at every predicted step rather than only the first. Under a sustained 20 N push toward a 0.06 m/0.35 m/s workspace/speed bound, predictive realization holds displacement to within 0.2-3.9 mm of the bound via a slack-relaxed soft constraint for both generators, while reactive clipping overshoots by 4.4 cm (impedance) and 8.8 cm (admittance). These results establish the architectural distinction on a planar plant and a real manipulator model, and motivate extension to nonlinear/passive generators, disturbance sweeps, and human experiments.

Keywords: physical human–robot interaction, generator–realizer architecture, model predictive control, behavior-realization separation, impedance control, admittance control, constraint handling.

1. Introduction

A pHRI controller has two conceptually different responsibilities:

1. specify how the robot should move in response to contact; and
2. decide what motion can safely be executed by the physical robot.

Hypothesis. These two responsibilities are independent problems: the specification of interaction behavior and the realization of that behavior under physical constraints can be solved by separate layers, without loss of optimality, for the affine generator class of Definition 1 (Section 3.1). Theorem 1 (Section 6) proves this for that class; Sections 7–8 test it on a planar plant and a real manipulator.

Conventional impedance control combines these responsibilities in one mass–spring–damper feedback law. Variable-impedance methods add adaptation, and model-predictive variants can optimize impedance parameters or jointly plan motion and compliance. This is useful, but the behavior representation and the constraint-handling controller remain tied to impedance parameters.

A concrete consequence of that coupling is actuator saturation. A conventional impedance or PD law computes joint torque directly from the interaction error, $\tau = J^\top (-K_d e - D_d \dot{e}) + \tau_{\text{ff}}$, where τ_{ff} is the feedforward gravity/Coriolis compensation and, on a redundant arm, the orientation-hold and null-space terms as well. Neither the impedance term nor τ_{ff} reasons about the actuator's torque limit. A stiff enough impedance, a large enough interaction force, or simply an unfavorable configuration in

which τ_{ff} itself is large can drive the commanded torque past τ_{max} with no warning that this was even possible. When it does, the actuator clips, and the acceleration the impedance law intended is silently not the acceleration the robot delivers — the same hidden-deviation failure the realization residual (Section 3.4) is built to expose for workspace and speed bounds, but for the actuator's own physical limit, which is the more fundamental constraint of the two: workspace and speed bounds are safety choices, torque limits are not. This paper's acceleration-based formulation keeps saturation under control by construction rather than by after-the-fact clipping: the horizon-wide torque constraint of Section 8.1 is checked against the *total* predicted torque — feedforward, orientation, and null-space terms included, not the interaction correction alone — at every step of the prediction, so saturation is anticipated rather than discovered, and when it is genuinely unavoidable it is reported as an explicit residual rather than absorbed silently by the actuator.

We study a different decomposition:

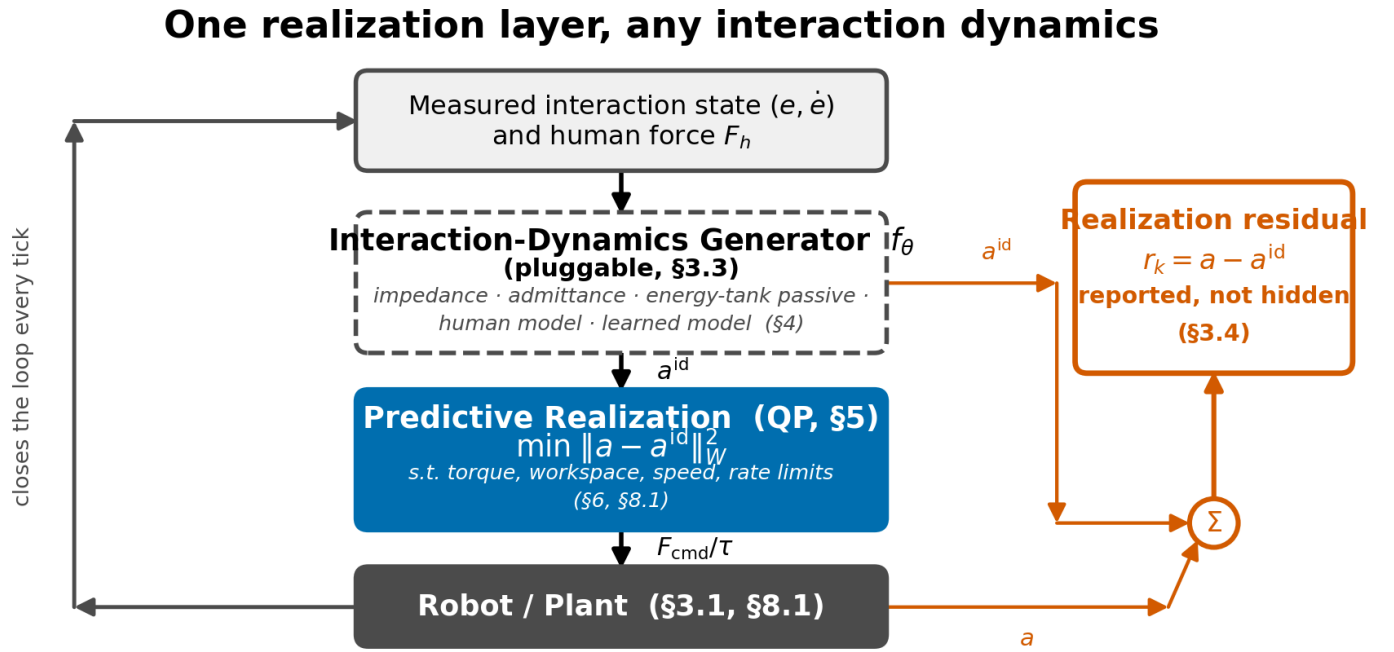


Figure 1. The generator (pluggable, dashed border) supplies a desired acceleration a^{id} ; the realization layer (blue) is a single QP that never refers to which generator produced it. The realization residual $r_k = a - a^{id}$ (right) is computed from the robot's actually realized acceleration and reported explicitly rather than absorbed silently into the command.

The generator expresses the intended interaction. The predictive layer is not asked to choose stiffness or damping. It chooses the actual robot command, and it is permitted to depart from the

reference dynamics only when required by physical constraints or secondary objectives. The magnitude and timing of that departure are explicit in the *realization residual*.

This is what the Hypothesis looks like as an architecture. The realization layer's cost, constraints, and decision variable are fixed by the robot and its safety limits alone; the only channel through which requested behavior enters is a single substitution — the generator's affine law (C_θ, G_θ) — into the cost's residual term. Section 6 states and proves this as the Affine Generator Independence theorem. One practical consequence: Section 4.1's impedance generator and Section 4.2's admittance generator are two validated instances of the same plug; Section 4.3 names three more — an energy-tank-modulated passive dynamics, a human internal (intent) model, and a learned interaction model — that fit the identical interface without touching the realization layer at all. Swapping the generator changes (C_θ, G_θ) (or, for a nonlinear generator, the map f_θ); every other line of the QP is untouched.

This separation changes both the algorithm and the scientific question. Instead of asking whether MPC can produce compliant tracking, we ask:

How accurately can a constrained robot realize a specified interaction dynamical system, and what controlled deviation is necessary when that system is temporarily incompatible with safety constraints?

The contributions of this draft are:

- a generator–realizer architecture in which the realization layer's cost, constraints, and decision variable are defined once, independent of which interaction dynamics is being realized, proved formally as an Affine Generator Independence theorem (Section 6);
- a convex predictive formulation for affine interaction dynamics, validated with impedance and force-guided admittance as two interchangeable instances of one generator interface;
- an explicit realization residual that quantifies how far the constrained robot had to depart from the specified dynamics, and why;
- a reproducible simulation, on both a planar point mass and a torque-controlled FR3 manipulator in MuJoCo, comparing horizon-wide constrained realization against instantaneous reference realization with identical command limits;
- an online generator-switching demonstration (Section 7.3) that exercises Theorem 1 directly: a single controller instance accepts a different generator mid-run, with no discontinuity beyond its own rate constraint.

We do **not** claim that embedding impedance behavior in MPC is itself new. Model predictive impedance control and model predictive interaction control already provide important prior formulations. The intended novelty is the generator-independence property itself: one realization layer,

proved once, that accepts any affine — and, prospectively, nonlinear — interaction-dynamics generator without redesign.

2. Related Work and Positioning

The closest literature is best distinguished by what the predictive or adaptive layer optimizes and by what quantity the robot-level controller is asked to reproduce.

Class	Behavior representation	Optimized robot-level variable	Role of prediction
Predictive variable impedance	M_d, D_d, K_d , often with a trajectory	Impedance parameters and/or trajectory	Select compliant behavior
Reference-model adaptive impedance	$M_d\ddot{e} + D_d\dot{e} + K_de = F_h$	Adaptive feedback parameters	Usually absent
Direct robot MPC	State or tracking-error dynamics	Force, torque, velocity, or acceleration	Optimize robot motion directly
Impedance/interaction MPC	Impedance, force, or coupled interaction model	Robot command and sometimes behavior parameters	Embed interaction objectives and constraints
Reference governor	A single reference/setpoint signal	Filtered reference fed to a fixed inner-loop controller	Supervise a non-predictive inner loop to enforce constraints
This work	$a^{\text{id}} = f_\theta(e, \dot{e}, F_h, z)$	Robot command u	Minimize dynamics-realization error under constraints

2.1 Predictive variable-impedance adaptation

Several approaches use MPC to adapt impedance parameters or to plan impedance and motion together. Anand et al. place a learned predictive policy above a low-level variable-impedance

controller [1]. Haninger et al. predict interaction and optimize trajectory and impedance online while incorporating safety constraints [2]. Recent predictive variable-impedance formulations continue this parameter-adaptation direction [3]. Their optimized behavior variables are stiffness, damping, or related trajectory parameters.

2.2 Reference-model impedance control

Model-reference adaptive impedance control specifies desired impedance dynamics and designs an adaptive nonlinear controller so that the robot approaches them [4]. This establishes the value of separating a desired model from the physical robot, but it does not by itself provide horizon-wide handling of coupled state and input constraints.

2.3 Direct robot MPC

A separate class applies MPC directly to the robot or to a feedback-linearized error model. Its decision variables are robot commands rather than impedance parameters. Such a controller can generate an effective closed-loop impedance, but unless a desired interaction model appears explicitly in the prediction objective, that impedance is an induced property of the optimizer rather than the behavior it is asked to realize. This distinction separates the present formulation from our earlier double-integrator tracking MPC: the earlier controller penalizes interaction error, whereas the present controller penalizes disagreement with an independently specified interaction dynamical system.

2.4 Model predictive impedance and interaction control

Bednarczyk et al. formulate impedance behavior through MPC and handle practical constraints such as velocity, energy, and jerk [5]. Gold et al. formulate model predictive interaction control using robot and interaction models in the optimal-control problem and express manipulation objectives through costs and constraints [6]. These papers are close precedents and make a broad “first constrained interaction-dynamics MPC” claim untenable.

2.5 Reference governors and supervisory constraint enforcement

Reference governors are the closest architectural relative to this paper's separation, from a different literature than Sections 2.1–2.4. A reference governor sits upstream of a fixed, already-designed inner-loop controller and modifies the reference signal fed to it — never the controller itself — so that constraints on the resulting closed-loop trajectory are respected; the foundational formulations are due to Bemporad, Kolmanovsky and Gilbert, and Garone et al. (*a specific reference-governor citation*

should be added here before submission.) The parallel is direct: a reference governor also separates "what behavior is commanded" from "what is safe to execute," and treats the former as replaceable without touching the latter. The difference is where prediction lives and what is exchanged. A reference governor typically supervises a fixed, non-predictive inner loop and manipulates a single reference signal; the realization layer here is itself the predictive, constraint-handling layer, and what is exchanged is not a signal but a complete affine (or, prospectively, nonlinear) dynamical law f_θ — a desired acceleration at any state and force, not a single setpoint. Reference governors also do not, in their standard form, expose a quantified residual between what was commanded and what the filtered system delivers; the realization residual (Section 3.4) is closer to that literature's constraint-violation margin than to its own outputs. For a reader arriving from that literature, this paper is best read as asking what a reference governor's separation looks like when the filtered object is a full behavioral dynamics rather than a setpoint, and the supervisor is itself predictive. Behavior-based robotics, task-space policy composition, and hierarchical behavior control share the broader theme of separating behavior specification from execution, though at a coarser grain — composing or arbitrating among multiple concurrent behaviors — than this paper's concern, which is realizing one behavior at a time under hard physical constraints; they are noted here for completeness rather than treated as close precedents.

2.6 Position of this work

Every row above except the last two optimizes a quantity tied to one behavior representation: impedance parameters, adaptive impedance gains, or a robot-level tracking error against a model that is impedance (or force, or a specific coupled interaction model) by construction. The reference-governor row (Section 2.5) is architecturally the closest relative, but it filters a single reference signal for a fixed, non-predictive inner loop rather than exchanging a complete behavioral dynamics with a predictive realization layer. Our distinction is not a better optimizer for any of these. It is that the desired interaction dynamics is exposed as a replaceable generator with a common output — a desired acceleration — and the optimizer's objective, constraints, and decision variable are defined against that common output rather than against impedance (or any other) behavior specifically. The same realization layer should accept impedance, admittance, passive energy-based dynamics, a human-intent model, or a learned model without changing the robot constraints, because none of those constraints ever mentioned impedance in the first place. This draft validates the distinction for two affine generators, impedance and admittance, precisely because they are behaviorally distinct enough (a position-restoring spring versus none) that one unmodified realization layer handling both is informative. Demonstrating the full breadth of the claim will require the nonlinear/passive generators of Section 4.3 and matched experiments against the baselines in this table.

3. Problem Formulation

3.1 General robot and contact-port dynamics

Consider a rigid manipulator

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) = \tau + J(q)^\top F_h,$$

where $q \in \mathbb{R}^{n_q}$, $\tau \in \mathbb{R}^{n_q}$, and F_h is the human wrench applied at the interaction port. For a task coordinate $y = h(q) \in \mathbb{R}^{n_y}$,

$$\dot{y} = J(q)\dot{q}, \quad \ddot{y} = J(q)\ddot{q} + \dot{J}(q, \dot{q})\dot{q}.$$

Substitution of the joint dynamics gives the affine acceleration map

$$\ddot{y} = b_y(q, \dot{q}, F_h) + G_y(q)\tau,$$

where

$$b_y(q, \dot{q}, F_h) = JM^{-1}(-C\dot{q} - g + J^\top F_h) + \dot{J}\dot{q},$$
$$G_y(q) = JM^{-1}.$$

Let y_d be a nominal interaction pose and define

$$e = y - y_d, \quad \dot{e} = \dot{y} - \dot{y}_d.$$

Definition 1 (interaction-dynamics generator). A generator is a causal map

$$\ddot{e}^{\text{id}} = f_\theta(e, \dot{e}, F_h, z),$$

where z denotes optional internal generator state and θ denotes behavior parameters. Its output is a desired contact-port acceleration, not a robot command. The generator is deliberately unaware of robot torque limits, joint limits, or workspace geometry; those belong to the realization layer.

The proposed architecture does not ask the generator to compute τ . Instead, it supplies $\ddot{e}^{\text{id}}$ above, and the predictive realization layer selects τ so that the physically realized $\ddot{e}$ approaches $\ddot{e}^{\text{id}}$ while satisfying robot constraints.

The full manipulator formulation is the target architecture. The executable study in this draft uses the following exactly discretized point-mass specialization so that the generator–realizer distinction can be isolated without kinematic or inverse-dynamics confounds.

3.2 Planar proof-of-concept plant

The proof-of-concept plant is a planar point mass:

$$m_r \ddot{p}(t) = u(t) + F_h(t),$$

where $p \in \mathbb{R}^2$ is displacement from a nominal interaction pose, $u \in \mathbb{R}^2$ is commanded robot force, $F_h \in \mathbb{R}^2$ is measured human force, and $m_r > 0$ is the realized robot mass. Define

$$x = \begin{bmatrix} p^\top & v^\top \end{bmatrix}^\top, \quad v = \dot{p}.$$

Under zero-order hold with period Δt ,

$$x_{k+1} = Ax_k + B(u_k + F_{h,k}),$$

$$A = \begin{bmatrix} I & \Delta t I \\ 0 & I \end{bmatrix}, \quad B = \begin{bmatrix} \frac{\Delta t^2}{2m_r} I \\ \frac{\Delta t}{m_r} I \end{bmatrix}.$$

The actual acceleration is

$$a_k = \frac{u_k + F_{h,k}}{m_r}.$$

3.3 The generator specialized to the planar plant

For the planar plant of Section 3.2, Definition 1 specializes with $x_k = [p_k^\top \ v_k^\top]^\top$ in place of $(e, \dot{e})$:

$$a_k^{\text{id}} = f_\theta(x_k, F_{h,k}, z_k).$$

For the affine class considered here,

$$a_k^{\text{id}} = C_\theta x_k + G_\theta F_{h,k}.$$

3.4 Realization residual

We define

$$r_k = a_k - a_k^{\text{id}}.$$

If $r_k = 0$, the robot exactly realizes the requested interaction dynamics at that sample. If $r_k \neq 0$, the robot executes the closest feasible dynamics selected by the optimizer. Reporting r_k distinguishes unavoidable safety intervention from failure to tune the reference model.

This definition is intentionally stronger than commanding a nominal impedance force and penalizing

$$\left\| u_k - u_k^{\text{imp}} \right\|_2^2.$$

The latter is controller-output tracking: it penalizes deviation from a nominal force command. $\|r_k\|_W^2$ is tracking too, but of a categorically different target — the *behavioral acceleration* the generator specifies, not a prior controller's output — so it compares the realized physical acceleration directly with the requested interaction dynamics. It remains meaningful when the generator is admittance-based, nonlinear, energy-based, or learned and no u_k^{imp} exists.

4. Configurable Generators

Section 3.1's Definition 1 specifies the generator interface abstractly; Section 3.3 specialized it to the planar plant. This section instantiates it. The point of listing five instances rather than the two this draft validates is to make the unification claim checkable rather than aspirational: each row below plugs into the identical realization layer of Section 5 by supplying a^{id} (or, for the affine rows, C_θ, G_θ), and none of them requires touching the QP's cost, constraints, or decision variable.

Generator	Requested dynamics	What changes in the QP	Status
Impedance (4.1)	$M_d a^{\text{id}} + D_d v + K_d p = F_h$	C_θ, G_θ only	Validated, Sections 7–8
Force-guided admittance (4.2)	$T_a a^{\text{id}} + v = Y F_h$	C_θ, G_θ only	Validated, Sections 7–8
Energy-tank passive dynamics (4.3)	$a^{\text{id}} = M_d^{-1}(F_h - D(v)v - \nabla V(p))$, tank-modulated	f_θ becomes nonlinear; QP unchanged if linearized per step	Sketched, not implemented
Human internal/intent model (4.3)	a^{id} from an estimated human control law	f_θ replaced by an estimator's output	Sketched, not implemented
Learned interaction model (4.3)	a^{id} from a fitted or trained model	f_θ replaced by the model; realization layer unaware of its origin	Sketched, not implemented

Nothing in this table is a different realization layer wearing a different name. It is the same Section 5 optimizer reading a different a^{id} .

4.1 Impedance generator

The desired impedance is

$$M_d a^{\text{id}} + D_d v + K_d p = F_h.$$

For isotropic parameters,

$$a^{\text{id}} = -\frac{K_d}{M_d} p - \frac{D_d}{M_d} v + \frac{1}{M_d} F_h.$$

Thus

$$C_{\text{imp}} = \begin{bmatrix} -K_d M_d^{-1} I & -D_d M_d^{-1} I \end{bmatrix}, \quad G_{\text{imp}} = M_d^{-1} I.$$

Unlike variable-impedance MPC (Section 2.1), M_d , D_d , K_d are not decision variables in the present formulation. They describe the requested behavior.

4.2 Force-guided admittance generator

The second generator requests a force-dependent velocity:

$$T_a a^{\text{id}} + v = Y F_h,$$

or

$$a^{\text{id}} = -T_a^{-1} v + T_a^{-1} Y F_h.$$

It has no position-restoring term. After force release, velocity decays and the displaced position is retained. This provides a qualitatively different behavior through the same acceleration interface.

4.3 Future nonlinear generators

The three remaining rows of Section 4's catalog table are not variants of one equation; each replaces the affine law differently. A nonlinear passive dynamics, with a general (not necessarily linear) damping term and a potential function in place of a linear spring,

$$a^{\text{id}} = M_d^{-1} (F_h - D(v)v - \nabla V(p)),$$

is one instance. An energy-tank-modulated version of the same idea adds a tank state that bounds the energy the generator may inject, rather than changing this expression's form. A human internal/intent model or a learned interaction model instead replace f_θ entirely with an estimator's or a trained model's output; neither has a closed form of this shape at all. In every case, a nonlinear

generator can be handled by sequential linearization, nonlinear MPC, or a sampled reference rollout, and the extension must preserve an auditable definition of the realization residual.

5. Predictive Realization

Section 4 supplies a^{id} (or, for the affine case, C_θ, G_θ). This section specifies the realization layer that consumes it — one condensed QP, unchanged across which generator is plugged in. At time k , let

$$U_k = \begin{bmatrix} u_{0|k}^\top & \cdots & u_{N-1|k}^\top \end{bmatrix}^\top.$$

The controller solves

$$\begin{aligned} \min_{U_k} \quad & \sum_{i=0}^{N-1} \|a_{i|k} - a_{i|k}^{\text{id}}\|_W^2 + \lambda_u \|u_{i|k}\|_2^2 + \lambda_{\Delta u} \|u_{i|k} - u_{i-1|k}\|_2^2 \\ \text{s.t.} \quad & x_{i+1|k} = Ax_{i|k} + B(u_{i|k} + \hat{F}_{h,i|k}), \\ & a_{i|k} = (u_{i|k} + \hat{F}_{h,i|k})/m_r, \\ & a_{i|k}^{\text{id}} = C_\theta x_{i|k} + G_\theta \hat{F}_{h,i|k}, \\ & |u_{i|k}|_\infty \leq u_{\max}, \\ & |u_{i|k} - u_{i-1|k}|_\infty \leq \dot{u}_{\max} \Delta t, \\ & |p_{i+1|k}|_\infty \leq p_{\max}, \\ & |v_{i+1|k}|_\infty \leq v_{\max}. \end{aligned}$$

Only $u_{0|k}^\star$ is executed. $u_{-1|k}$ in the $i = 0$ rate penalty and rate constraint denotes the command actually applied at the previous solve, carried by the controller between calls (zero before the first solve), so both are well-defined from the very first call. The implementation uses the measured force held constant across the horizon, $\hat{F}_{h,i|k} = F_{h,k}$, rather than oracle knowledge of the scripted force. A learned or physics-based force predictor can be substituted without changing the generator interface.

5.1 Tracking behavioral acceleration, not a motion trajectory

The objective $\|a_{i|k} - a_{i|k}^{\text{id}}\|_W^2$ is tracking — of a categorically different target than a position or velocity trajectory. No desired position trajectory is constructed; the optimizer instead compares two accelerations evaluated at the predicted interaction state: the acceleration the constrained robot will produce and the behavioral acceleration the interaction model requests. Position and velocity enter as generator states and as safety-constrained quantities, never as a tracked reference in their own right.

5.2 The decision variable is the robot command, not the generator's parameters

The optimization variable is the robot command sequence U_k . The generator parameters θ are fixed during each experiment. Switching from impedance to admittance changes C_θ, G_θ , not the QP constraints or the command variable — stated formally as Theorem 1 (Section 6).

6. Basic Properties

Section 1 asserted, and Section 5.2 restated, that swapping the generator changes nothing in the QP except (C_θ, G_θ) . This section states that claim as a theorem and proves it, then establishes what the resulting QP guarantees in principle — convexity, exact realization when unconstrained, one-step constraint satisfaction — and, Proposition 3 in particular, what it does not.

Theorem 1 (Affine Generator Independence). Fix the plant (A, B, m_r) , horizon N , weights $(W, \lambda_u, \lambda_{\Delta u})$, and bounds $(u_{\max}, \dot{u}_{\max}, p_{\max}, v_{\max})$. There is a single map $(C, G) \mapsto Q(C, G)$ — a quadratic-program template depending only on the plant, horizon, weights, and bounds above, never on θ — such that, for every generator θ with affine law (C_θ, G_θ) , the realization QP of Section 5 is exactly $Q(C_\theta, G_\theta)$. Consequently, for any two generators θ_1, θ_2 , the instances $Q(C_{\theta_1}, G_{\theta_1})$ and $Q(C_{\theta_2}, G_{\theta_2})$ share the same decision variable U_k and the same feasible set, and differ only in the coefficients Q contributes from (C_θ, G_θ) — never in the variable, the constraint set, or the functional form of the objective. Realizing a different generator therefore requires changing only the argument (C_θ, G_θ) to Q , nothing else. This is stated for the affine generator class of Definition 1; a nonlinear generator (Section 4.3) makes $a_{i|k}^{\text{id}}$ a nonlinear function of U_k , so $\|r_{i|k}\|_W^2$ is no longer guaranteed convex quadratic and the template Q does not apply without per-step linearization.

Proof. Every constraint in Section 5's QP — the dynamics recursion, the acceleration definition, and the four box constraints — is written in terms of $A, B, m_r, u_{\max}, \dot{u}_{\max}, p_{\max}, v_{\max}, U_k$, and the resulting state/acceleration sequence; none references θ, C_θ , or G_θ . The feasible set is therefore a fixed polyhedron, identical for every generator. The only appearance of (C_θ, G_θ) anywhere in the problem is the term $a_{i|k}^{\text{id}} = C_\theta x_{i|k} + G_\theta \hat{F}_{h,i|k}$ inside the primary cost, entering exactly once per horizon step as an affine substitution into $r_{i|k} = a_{i|k} - a_{i|k}^{\text{id}}$. Because $x_{i|k}$ is itself independent of (C_θ, G_θ) and affine in U_k (established in Proposition 1's proof below), $a_{i|k}^{\text{id}}$ is affine in U_k with (C_θ, G_θ) -dependent coefficients only, so $\|r_{i|k}\|_W^2$ is a convex quadratic in U_k whose Hessian and gradient contributions are determined entirely by (C_θ, G_θ) ; the secondary terms $\lambda_u \|u_{i|k}\|_2^2$ and $\lambda_{\Delta u} \|u_{i|k} - u_{i-1|k}\|_2^2$ do not involve (C_θ, G_θ) at all. Collecting these pieces defines $Q(C, G)$ as a function of (C, G) alone — the plant, horizon, weights, and bounds enter Q as fixed parameters, not arguments — and by construction $Q(C_\theta, G_\theta)$ is exactly the QP Section 5 poses for generator θ . ■

This is why Section 4's five-instance catalog is checkable rather than aspirational: each affine row supplies a different (C_θ, G_θ) into the one template Q above, and the theorem is precisely the statement that nothing else needs to change. The nonlinear rows of Section 4.3 are future work for exactly the reason the theorem's affine hypothesis makes explicit: without per-step linearization, f_θ does not factor through a fixed (C, G) pair, and Q does not apply.

Proposition 1 (convexity for affine generators). If $W \succeq 0$, $\lambda_u \geq 0$, and $\lambda_{\Delta u} \geq 0$, then the finite-horizon problem in Section 5 is a convex quadratic program. It is strictly convex if the assembled Hessian is positive definite, including the common case $\lambda_u > 0$.

Proof. The lifted state $x_{i|k}$ is affine in U_k by the recursion $x_{i+1|k} = Ax_{i|k} + B(u_{i|k} + \hat{F}_{h,i|k})$ unrolled from $x_{0|k}$. Hence $a_{i|k} = (u_{i|k} + \hat{F}_{h,i|k})/m_r$ and $a_{i|k}^{\text{id}} = C_\theta x_{i|k} + G_\theta \hat{F}_{h,i|k}$ are affine in U_k , so $r_{i|k} = a_{i|k} - a_{i|k}^{\text{id}}$ is affine in U_k and $\|r_{i|k}\|_W^2$ is convex quadratic since $W \succeq 0$. The regularization terms $\lambda_u \|u_{i|k}\|_2^2$ and $\lambda_{\Delta u} \|u_{i|k} - u_{i-1|k}\|_2^2$ are convex quadratic in U_k for $\lambda_u, \lambda_{\Delta u} \geq 0$, and a sum of convex functions is convex. Every listed constraint bounds an affine function of U_k in absolute value, so the feasible set is an intersection of half-spaces, hence a convex polyhedron. ■

Proposition 2 (exact unconstrained realization). Assume additionally that $W \succ 0$ (strictly positive definite — a stronger hypothesis than Proposition 1's $W \succeq 0$). Suppose an input sequence exists for which $r_{i|k} = 0$ over the horizon and no constraint is active. If the realization residual is optimized lexicographically before secondary effort objectives—or equivalently the secondary weights are zero—then every optimal primary solution realizes the reference dynamics exactly.

Proof. The primary objective $\sum_i \|r_{i|k}\|_W^2$ is nonnegative and attains the value zero at the assumed feasible sequence, so its minimum over the horizon is zero. Under lexicographic priority (or zero secondary weights), any optimal solution must attain this minimum, so $\sum_i \|r_{i|k}\|_W^2 = 0$, and since each term is nonnegative, $\|r_{i|k}\|_W^2 = 0$ for every i . Because $W \succ 0$, a quadratic form $\|r\|_W^2$ vanishes only at $r = 0$; if W were only positive semi-definite, this step would give $r_{i|k} \in \ker W$ rather than $r_{i|k} = 0$, which is why the stronger hypothesis is needed here but not in Proposition 1. ■

With finite scalar weights, effort regularization can intentionally introduce a small residual even when exact realization is feasible. The present experiment uses small, not zero, secondary weights, so the predictive controller's measured residual is not exactly zero even during unconstrained intervals (Table 1) — this is the finite-weight case Proposition 2 itself distinguishes from the idealized zero-weight one, not a violation of it. The reactive comparator applies the generator's instantaneous law directly, with no horizon and no secondary regularization to trade against; its residual sits at numerical-solver tolerance (Table 1) for the simpler reason that it is solving a different, unconstrained algebraic inversion, not the same QP with different weights. That near-exactness is a check that the underlying generator and dynamics model are implemented correctly, not a demonstration of Proposition 2 itself.

Proposition 3 (one-step constraint inheritance). Assume the model and current human force are exact over the executed sample and the QP is feasible. Then $u_{0|k}^*$, $p_{1|k}$, and $v_{1|k}$ satisfy their corresponding QP bounds. Re-solving at every sample yields constraint satisfaction by induction while feasibility is retained.

This proposition is not a recursive-feasibility theorem. A deployable version requires a terminal invariant set, soft constraints with a quantified fallback, or a backup safe controller. Sudden force changes inside one sample and model error must also be covered by robust constraint tightening.

7. Simulation

Section 6 establishes what the QP guarantees in principle — convexity, exact realization when unconstrained, one-step constraint satisfaction, and what it does not guarantee (recursive feasibility). This section tests those properties empirically on the planar plant of Section 3.2, comparing predictive realization against the reactive comparator under identical command and rate limits.

7.1 Reproducible setup

The simulation uses a 2.5 kg planar point mass, $\Delta t = 0.02$ s, and a 20-step (0.4 s) horizon. A smooth 12 N force is applied along $+y$ from 1 s to 3 s. Limits are

$$|u_j| \leq 18 \text{ N}, \quad |v_j| \leq 0.22 \text{ m/s}, \quad |p_j| \leq 0.10 \text{ m},$$

with $|\dot{u}_j| \leq 180 \text{ N/s}$. The realization, force, and force-rate weights are 1, 2×10^{-4} , and 10^{-3} , respectively.

The impedance generator (Section 4.1) uses

$$M_d = 2.0 \text{ kg}, \quad D_d = 18 \text{ Ns/m}, \quad K_d = 45 \text{ N/m}.$$

The admittance generator (Section 4.2) uses $T_a = 0.25$ s and $Y = 0.025 \text{ m/(Ns)}$.

The comparator computes the instantaneous reference acceleration, converts it to robot force, and applies the same 18 N force and 180 N/s slew limits. It has no prediction of position or speed constraints. This comparator is intentionally minimal: it isolates what horizon-wide state constraints change; it is not presented as a survey-complete baseline.

7.2 Results

One predictive safety layer realizes two reference interaction dynamics

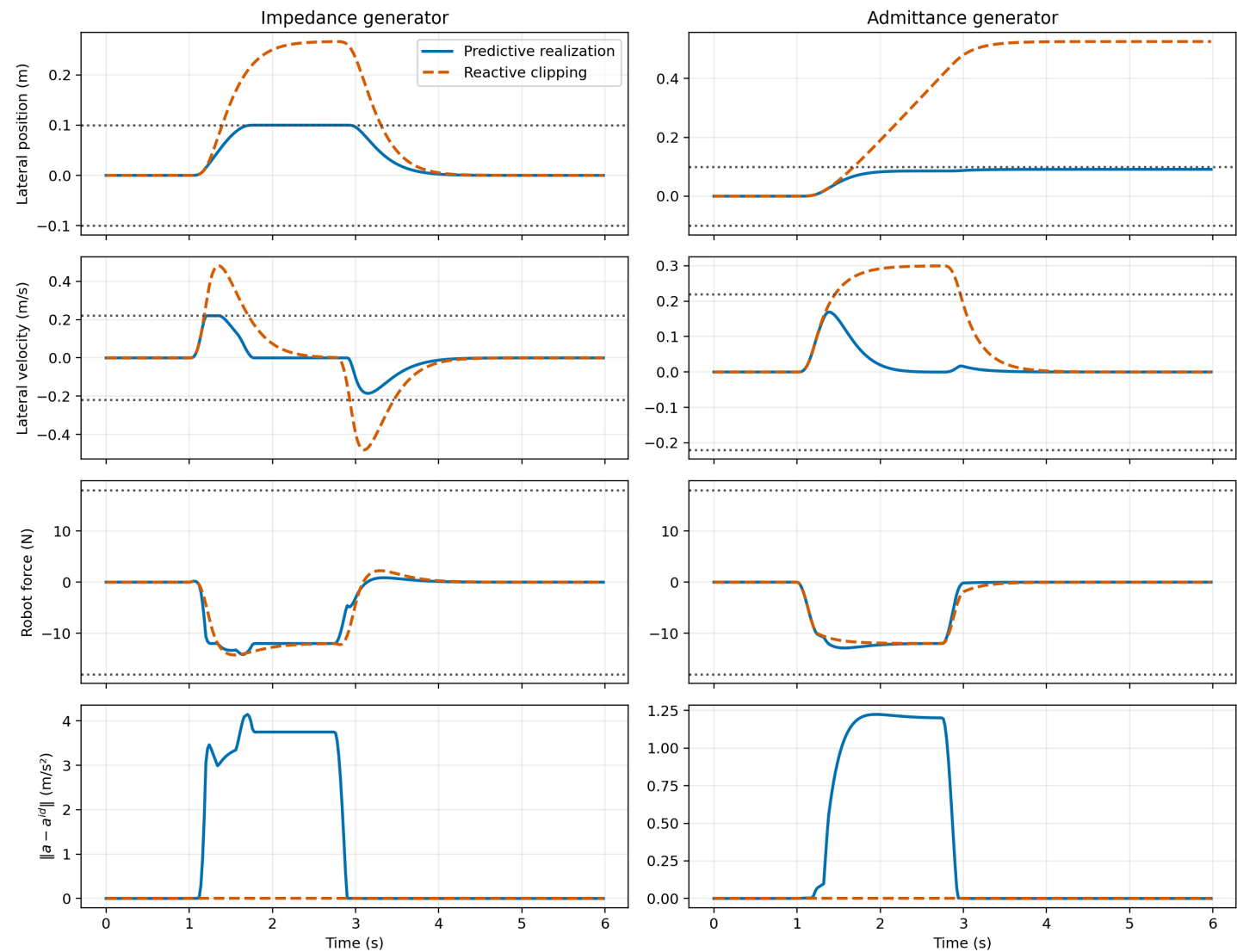


Figure 2. The same predictive realization layer is used with impedance (left) and force-guided admittance (right). Dotted lines are component-wise limits. Reactive clipping follows the desired dynamics almost exactly but does not anticipate state-limit violations. Predictive realization departs from the generator when necessary, visible in the final row.

Generator	Controller	Realization RMSE (m/s ²)	Peak $ p_j $ (m)	Peak speed (m/s)	Peak $ u_j $ (N)	State-limit violation
Impedance	Predictive realization	1.351	0.100002	0.220	14.16	0.000002 m numerical tolerance

Generator	Controller	Realization RMSE (m/s ²)	Peak $ p_j $ (m)	Peak speed (m/s)	Peak $ u_j $ (N)	State-limit violation
Impedance	Reactive clipping	$< 10^{-6}$	0.2663	0.480	14.24	0.1663 m, 0.260 m/s
Admittance	Predictive realization	0.405	0.0912	0.169	12.88	none
Admittance	Reactive clipping	$< 10^{-6}$	0.5250	0.300	12.00	0.4250 m, 0.080 m/s

Table 1. State-limit violation reports the amount by which the 0.10 m / 0.22 m/s workspace/speed bound is exceeded. Realization RMSE for the reactive comparator is at numerical-solver tolerance because it directly inverts the generator's instantaneous law for the force that realizes it exactly, with no horizon or secondary regularization to trade against; it is not zero for predictive realization because the horizon-wide formulation trades a small residual for constraint satisfaction and finite secondary weights (Proposition 2, Section 6), and because the reported metric aggregates over the whole run, not only the constrained interval.

The impedance reference has a static displacement $F_h/K_d = 12/45 = 0.267$ m, matching the 0.266 m simulated peak after the smooth force ramp. Because this behavior is incompatible with the 0.10 m workspace, the predictive controller increases the realization residual while the bound is active. After release, it returns to the impedance equilibrium at the origin.

The admittance reference requests approximately 0.30 m/s under 12 N and has no restoring spring. Reactive realization therefore accumulates 0.525 m displacement and retains it after release. The constrained controller begins departing from the requested velocity dynamics before reaching the workspace boundary and settles at 0.091 m. The generator remains unchanged; the behavior modification is entirely attributable to the realization layer.

7.3 Online generator switching

Sections 7.1–7.2 validate the generator interface by running two separate simulations, one per generator — informative, but not yet a demonstration that a single running controller accepts a different generator without redesign. This section tests that directly. One `InteractionDynamicsMPC` instance is constructed once, with the impedance generator; a small constant lateral force (1 N, well inside every bound) is applied for the entire 6 s run, with no release. At $t = 2$ s and $t = 4$ s, exactly one attribute is reassigned on the live controller — `controller.generator` — switching impedance

→ admittance → impedance; nothing else (the QP weights, constraints, decision variable, or the `previous_command` state carried between solves) is touched or reconstructed.

Online generator switching: one controller instance, only `.generator` reassigned

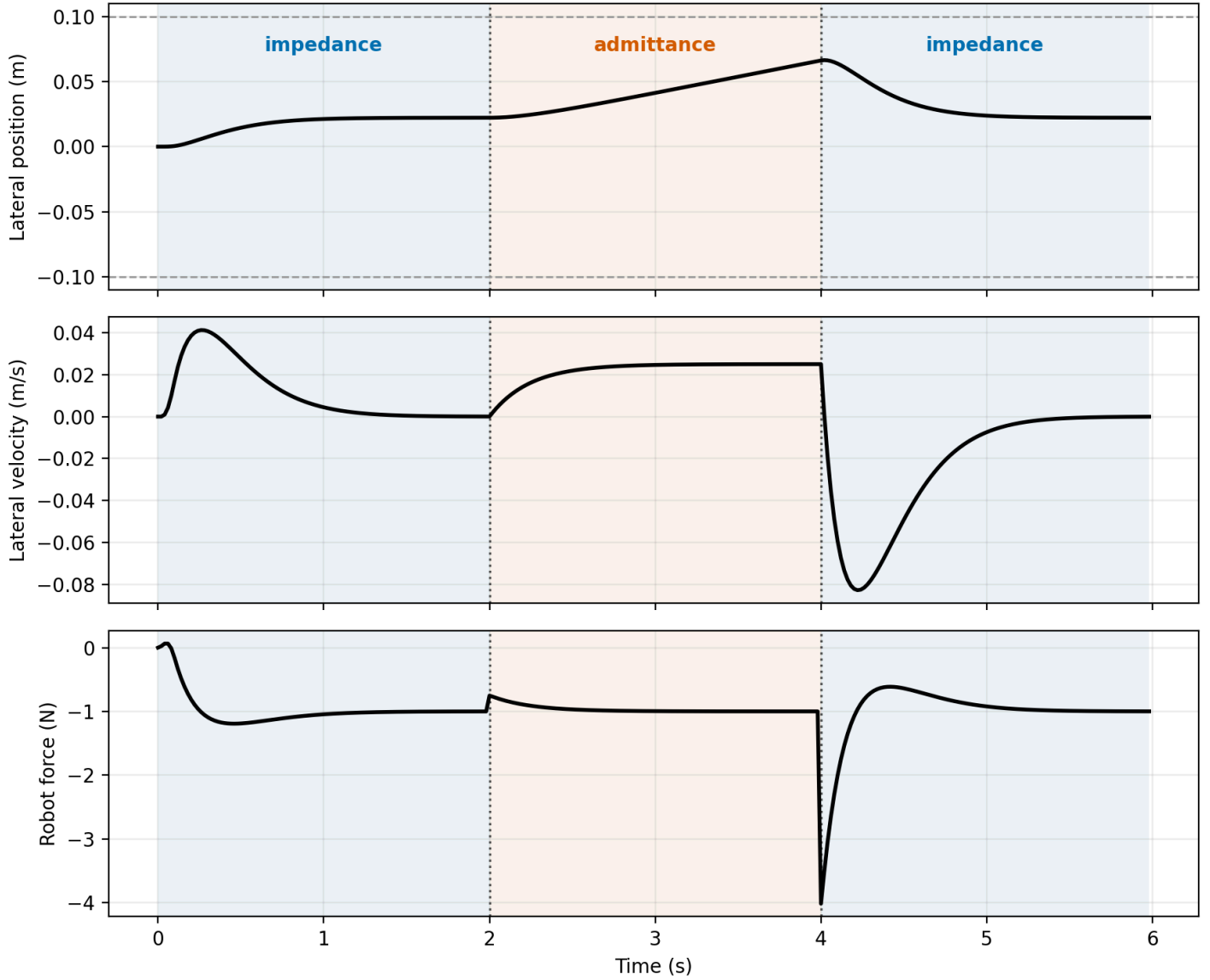


Figure 3. One controller instance; only `.generator` is reassigned at $t = 2$ s and $t = 4$ s (dotted lines). Position converges to the impedance equilibrium $F_h/K_d \approx 0.022$ m during both impedance segments, drifts at the admittance generator's steady-state velocity $Y F_h = 0.025$ m/s with no restoring term during the middle segment, and smoothly re-converges to the same equilibrium after re-entering impedance from a very different state (nonzero position and velocity) at $t = 4$ s.

Both switches are smooth: the largest command-force jump at a switch instant (3.02 N at $t = 4$ s, the harder of the two transitions) stays within the QP's own 3.6 N per-tick rate limit — the same constraint that already bounds every other command-rate step in the run, not a special case added for the switch. Position and speed stay within 0.067 m and 0.083 m of the origin throughout, well inside the

0.10 m / 0.22 m/s bounds; the demonstration is deliberately kept away from the workspace boundary so it is not confounded with Section 7.2's constraint-handling story. The second impedance segment is the more informative of the two transitions: it must re-converge to the impedance equilibrium starting from wherever the admittance segment left the system — a different position and a nonzero velocity — and does so without any special-casing on re-entry.

7.4 What the experiments establish

The results support four limited claims:

1. two qualitatively different affine interaction models use the same constrained predictive implementation;
2. an explicit dynamics residual identifies when safety requires deviation from the requested behavior;
3. horizon-wide state constraints prevent violations that command clipping alone cannot prevent in this deterministic model;
4. a single controller instance accepts a different generator online, mid-run, by reassigning one attribute, with no discontinuity beyond what its own rate constraint already permits.

They do not establish manipulator-level feasibility, coupled human–robot stability, passivity, robustness to force-estimation delay, or real-time performance on embedded hardware.

8. FR3 Manipulator Validation

Section 7 isolates the generator–realizer distinction from kinematic and inverse-dynamics confounds by using a point mass. This section instantiates the same architecture on a torque-controlled 7-DOF Franka FR3 simulated in MuJoCo, replacing the exact double-integrator plant with the real, configuration-dependent manipulator dynamics of Section 3.1.

8.1 Architecture

At each solve, let $J_v(q) \in \mathbb{R}^{3 \times 7}$ be the translational Jacobian and $\Lambda(q)^{-1} = J_v M(q)^{-1} J_v^\top$ the (regularized) inverse operational-space inertia for the position coordinate.

Full closed-loop law. Let R_d be the held reference orientation, $e_R(q) \in \mathbb{R}^3$ the corresponding orientation error, $J_w(q) \in \mathbb{R}^{3 \times 7}$ the rotational Jacobian, and q_0 a fixed neutral configuration. The commanded joint torque is assembled from four terms:

$$\begin{aligned}
\tau_{\text{ff}}(q, \dot{q}) &= C(q, \dot{q})\dot{q} + g(q) && \text{(feedforward)} \\
\tau_{\text{orient}}(q, \dot{q}) &= J_w(q)^\top (-K_{\text{rot}} e_R(q) - D_{\text{rot}} \omega) && \text{(orientation hold, raw)} \\
\tau_{\text{null}}(q, \dot{q}) &= -k_{\text{null}}(q - q_0) - d_{\text{null}}\dot{q} && \text{(null-space centering, raw)} \\
\bar{N}_v(q) &= I_7 - M(q)^{-1} J_v(q)^\top \Lambda(q) J_v(q) && \text{(position-consistent null-space projector)} \\
\tau_{\text{aux}}(q, \dot{q}) &= \bar{N}_v(q)^\top [\tau_{\text{orient}}(q, \dot{q}) + \tau_{\text{null}}(q, \dot{q})] && \text{(projected, see below)} \\
\tau_{\text{base}}(q, \dot{q}) &= \tau_{\text{ff}}(q, \dot{q}) + \tau_{\text{aux}}(q, \dot{q})
\end{aligned}$$

and the executed torque is

$$\tau = \tau_{\text{base}}(q, \dot{q}) + J_v(q)^\top F_{\text{cmd}}.$$

Define $d_{\text{known}}(q, \dot{q}) = J_v(q)M(q)^{-1}\tau_{\text{aux}}(q, \dot{q})$, the effect of τ_{aux} — the non-feedforward part of τ_{base} — on translational acceleration. With this, the residual translational dynamics used by the QP are

$$\ddot{e} = \Lambda(q)^{-1}(F_{\text{cmd}} + F_h) + d_{\text{known}},$$

which corrects the sign of the sketch in the earlier draft of this section: the realized acceleration is *positive* in the commanded force, matching the point-mass convention of Section 3.2. $F_{\text{cmd}} \in \mathbb{R}^3$, the QP's Cartesian correction force, is supplied by whichever controller is active. The predictive controller sets $F_{\text{cmd}} = F_{0|k}^*$, the first element of the receding-horizon solution (Section 5, condensed later in this section). The reactive comparator instead sets

$$F_{\text{cmd}} = \text{clip}\left(\Lambda(q)[a^{\text{id}}(x, F_h) - d_{\text{known}}] - F_h\right),$$

rate- and magnitude-limited to the same bounds as the predictive controller, with $a^{\text{id}} = C_\theta x + G_\theta F_h$ the generator's instantaneous desired acceleration (Section 4). Both controllers share the identical τ_{base} and the identical torque-assembly step; they differ only in how F_{cmd} is chosen.

$\bar{N}_v(q)$ above is not the only choice that projects the null-space torque out of the way, and getting this projection right mattered in practice. A first implementation projected the null-space torque through the null-space projector of the *full* 6-row Jacobian (position and orientation stacked) and left the orientation-hold torque unprojected. Neither choice is null-space-orthogonal to the position task alone, so both leaked into $\ddot{e}$; d_{known} was computed to cancel that leakage in the QP's model. Under sustained interaction this produced a positive-feedback instability: leakage grows the joint configuration's deviation from the neutral posture, which grows the leakage, which grows the compensating force applied through the same unprojected torque path, which leaks further — an unbounded drift confirmed empirically and insensitive to retuning the orientation or null-space gains (ruling out a tuning fix). $\bar{N}_v(q)$ — the dynamically-consistent null-space projector of the position Jacobian alone (Khatib 1987), applied to *both* the orientation-hold and null-space torques rather than

to the null-space term only — is the fix: it guarantees $J_v M^{-1} \bar{N}_v^\top(\cdot) \equiv 0$ by construction, which makes d_{known} approximately zero rather than merely compensated for, and removes the feedback path.

The realization QP freezes $\Lambda(q)^{-1}$, $\tau_{\text{base}}(q, \dot{q})$, and $J_v(q)$ at the configuration q_k of the current solve and holds them across the horizon — the same frozen-Jacobian approximation used elsewhere for manipulator MPC under a fixed per-solve linearization. This keeps the per-solve problem an exact convex QP in the condensed correction-force sequence, at the cost of being a local approximation rather than an exact prediction of future feasibility (the same caveat Proposition 3 already states for the point-mass case). Per-joint torque feasibility, $|\tau_{\text{base}} + J_v^\top F_{\text{cmd},i}| \leq \tau_{\text{max}}$, is enforced as a hard constraint at every horizon step i , not only $i = 0$: a first-step-only formulation can pass an aggregate feasibility check while the QP's own internal plan still relies on later steps the actuators cannot deliver.

Note that this bound is on the *sum* $\tau_{\text{base}} + J_v^\top F_{\text{cmd},i}$, not on $F_{\text{cmd},i}$ alone: τ_{base} is not a fixed, negligible offset, and in an unfavorable configuration (near a joint limit, or during an orientation-hold transient) it can itself consume a substantial share of τ_{max} , leaving less budget for the interaction correction than a constraint on $F_{\text{cmd},i}$ alone would assume. This is precisely the actuator-saturation risk motivated in Section 1: the constraint here is written against the total predicted torque so that saturation is a modeled, anticipated event rather than something the actuator's own clipping discovers after the fact. Concretely, the bound is part of the QP's feasible set, not a check applied to its output: any F_{cmd} sequence the solver returns satisfies it by construction, because an infeasible sequence is not a candidate solution in the first place. This is the mechanistic difference from a PD or impedance law, which evaluates $\tau = J^\top(-K_d e - D_d \dot{e}) + \tau_{\text{ff}}$ directly and hands it to the actuator; if that value exceeds τ_{max} , something downstream truncates it, and the acceleration the law intended silently stops being the acceleration delivered. Here, when the generator's desired acceleration would require more torque than the budget allows, the optimizer instead returns the closest feasible F_{cmd} — trading realization accuracy against feasibility — and that trade-off is exactly the realization residual r_k of Section 3.4: reported, not hidden.

The Cartesian workspace and speed box is instead slack-relaxed with a large penalty (10^8): under the frozen-Jacobian approximation, a hard box constraint has no recursive-feasibility guarantee against the true nonlinear MuJoCo dynamics, and this was confirmed empirically — tightening or loosening the bound, or loosening the torque limit, did not remove solver infeasibility near an active bound, isolating the hard box (not the torque constraint) as the cause. The slack penalty keeps every solve well-defined while still making violation costly relative to the realization objective. On the rare solves where the QP is still reported infeasible, the controller falls back to the reactive/clipped instantaneous law (never to zero correction, which was tried first and produced a cascading runaway) so that some corrective force is always applied.

The fast control loop recomputes τ_{base} , the orientation term, and the null-space term from the current $(q, \dot{q})$ at 1 kHz; the QP is re-solved only every 20 ms (a 50 Hz outer loop), holding F_{cmd} between solves. Caching the full joint torque at the outer-loop rate, rather than only the slow correction term, would confound any comparison of predictive against reactive control at a common inner-loop rate.

8.2 Benchmark scenario

Mirroring the planar prototype's own story rather than a separate benchmark design: the EE holds a fixed nominal pose and orientation while a sustained human push toward a workspace/speed boundary is applied, and predictive realization is compared against reactive clipping under an identical command/rate box. Neither controller has oracle knowledge of the scripted force; both receive a zero-order-hold forecast of the currently measured force, as in Section 7.

The push is a 20 N force along $-z$ with a raised-cosine ramp from $t = 1.0$ s to 1.25 s, a hold to $t = 1.75$ s, and a symmetric ramp-down to $t = 2.0$ s. It is applied physically to the MuJoCo end-effector body, not only supplied to the controller's internal model, so the comparison reflects what the real simulated dynamics do under the push, not only what each controller believes is happening.

The workspace/speed bound is

$$|e_z| \leq 0.06 \text{ m}, \quad |v| \leq 0.35 \text{ m/s},$$

the QP horizon is 15 steps (0.3 s) at $\Delta t = 0.02$ s, and per-joint torque limits match the FR3 (± 87 Nm for joints 1–4, ± 12 Nm for joints 5–7).

The impedance generator (Section 4.1) uses $M_d = 2.0$ kg, $D_d = 28$ Ns/m, $K_d = 200$ N/m. The admittance generator (Section 4.2) uses $T_a = 0.3$ s and $Y = 0.01$ m/(Ns). Each condition runs for 6 s of simulated time.

8.3 Results

One predictive realization layer on a real FR3: two interaction dynamics, horizon-wide torque feasibility

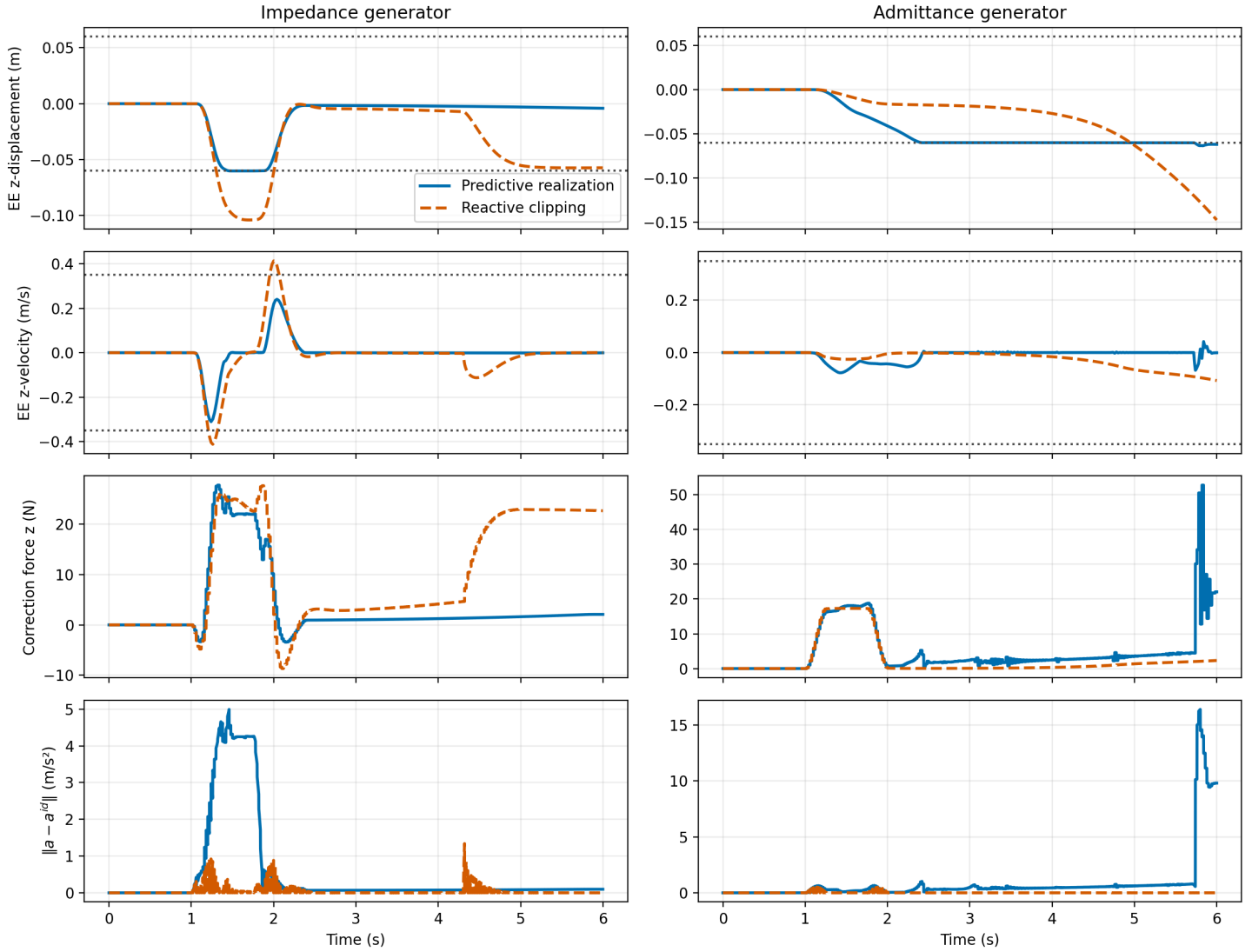


Figure 4. The same predictive realization layer on a real FR3, impedance (left) and admittance (right) generators, predictive realization (solid) against reactive clipping (dashed). Dotted lines are the workspace/speed bounds. Predictive realization tracks the bound closely for the duration of the push; reactive clipping overshoots substantially, and for the admittance generator does not recover the displacement after force release.

Generator	Controller	Realization RMSE (m/s ²)	Peak $ e_z $ (m)	Peak speed (m/s)	Peak $ \tau $ (Nm)	State- limit violation	Solve time mean / max (ms)
Impedance	Predictive realization	1.32	0.0602	0.313	32.81	0.0002 m	3.8 / 6.8
Impedance	Reactive clipping	0.127	0.1040	0.414	32.44	0.0440 m, 0.064 m/s	—
Admittance	Predictive realization	2.52	0.0639	0.104	35.11	0.0039 m	41.7 / 136.8
Admittance	Reactive clipping	0.061	0.1476	0.114	27.37	0.0876 m	—

Table 2. Peak $|\tau|$ is the largest magnitude reached by any joint, well within the FR3's 87 Nm (joints 1–4) / 12 Nm (joints 5–7) limits in every condition; state-limit violation reports the amount by which the 0.06 m / 0.35 m/s workspace/speed bound is exceeded, in the same format as the planar study's table in Section 7.2. The reactive comparator does not solve a QP, so its solve-time column is empty rather than zero.

No condition violates the per-joint torque limit at the executed sample or at any predicted horizon step (confirmed by direct inspection of the horizon-wide predicted-torque sequence, not only the aggregate maximum), and no solve is reported infeasible in this scenario.

The impedance reference has an unconstrained static displacement $F_h/K_d = 20/200 = 0.10$ m, which by itself already exceeds the 0.06 m bound. Reactive clipping tracks toward that equilibrium and peaks at 0.104 m (slightly above the static value, from the same ramp-overshoot effect as the planar case in Section 7.2), overshooting the bound by 4.4 cm. Predictive realization instead departs from the requested dynamics while the bound is active, holding displacement to 0.0602 m.

The admittance reference has no equilibrium displacement to compare against, only a steady-state velocity $Y F_h = 0.01 \times 20 = 0.2$ m/s while the force is held, with no restoring term to pull it back after release. Reactive clipping accumulates displacement through the hold and continues briefly after force release before the velocity decays (time constant $T_a = 0.3$ s), peaking at 0.1476 m — an 8.8 cm overshoot — and never recovers it, since the generator itself has no position-restoring term (Section 4.2) and the reactive comparator has no lookahead to anticipate the boundary. Predictive

realization begins departing from the requested velocity before the bound is reached and settles at 0.0639 m.

Two limitations are visible in the results and worth reporting rather than hiding.

- First, the admittance-generator QP's solve time (mean 41.8 ms, max 136.8 ms) exceeds the nominal 20 ms outer-loop period; the impedance generator's solves (mean 3.8 ms) do not. Both use an identical QP structure and an un-warm-started OSQP solve each period, so the difference is attributable to problem conditioning under the admittance generator's dynamics rather than problem size; closing this gap is future work; the outer loop in this study is data-rate, not wall-clock, paced, so the reported numbers are indicative of solver effort rather than an achieved real-time guarantee.
- Second, in the admittance condition, once the EE has drifted close to the 0.06 m bound for an extended period (a consequence of the admittance generator's lack of a restoring force under this benchmark's longer hold), the slack variable begins to chatter over a short late-time window (visible in Figure 4's bottom-right panel near $t \approx 5.7\text{--}6.0$ s): bounded, not divergent, but a direct empirical instance of Proposition 3's statement that this formulation offers no recursive-feasibility guarantee near a persistently active bound.

8.4 Scope and what remains

This is a focused core, not the full manipulator validation program. The generator set is impedance and admittance only, not the nonlinear/passive generator of Section 4.3; the scenario is a single sustained push, not a sweep over delay, sensor noise, or mass mismatch; there is no collision constraint, and no comparison against the classical-impedance, reactive-admittance, predictive-variable-impedance, model-predictive-impedance, or LQ-interaction-error baselines named in Section 2. The online generator-switching demonstration (Section 7.3) is planar only; an FR3 counterpart, switching generators on the live `FR3RealizationMPC` instance mid-run, remains future work.

Human-participant and hardware validation remain future work, as does closing the admittance solve-time gap identified in Section 8.3.

9. Limitations and Open Theory

The results above establish the architectural distinction on two plants; they do not establish everything a mature interaction-dynamics realization framework would need. Five gaps are worth stating plainly rather than leaving implicit.

First, the current generator class is affine and memoryless (Section 4.3 sketches richer instances, none yet implemented). A stateful or learned generator requires augmented prediction and a clear treatment of model uncertainty.

Second, the realization cost does not itself imply passivity. A passive reference generator can still be rendered non-passive by constraint-induced deviation or secondary optimization. A useful extension is a dissipativity constraint on the realized port variables together with an energy tank or passivity observer/controller.

Third, hard state constraints can become infeasible after an unpredicted impulse — Proposition 3 already flags this as a gap, not a guarantee. The FR3 study (Section 8) addresses it operationally with a slack-relaxed workspace/speed box and a reactive fallback on solver infeasibility, and both were exercised without producing unbounded violations in the tested scenario — but this is an empirical demonstration for one scenario, not the terminal invariant set, quantified soft-constraint fallback, or backup safe controller Proposition 3 says a deployable version would need. Softening safety-critical constraints without defining the resulting risk remains not sufficient on its own.

Fourth, the interaction-dynamics residual has physical units and depends on the chosen output coordinates, and the FR3 study only exercises it on the translational coordinate. Orientation on the FR3 is held by a fixed PD law, τ_{orient} (Section 8.1), not by a generator: it is conventional impedance control of exactly the kind Section 1 argues against as the object of study, present here because Section 4's generator interface has not yet been extended to orientation. The "one generator, one realization layer, for any interaction dynamics" claim is therefore validated for translation only. Extending the generator interface to orientation, and comparing translational and rotational behavior under one residual once that exists, will require a principled metric or energy-weighted normalization across coordinates with different units. A related caveat applies even within one coordinate: Tables 1 and 2 report raw RMSE across generators with different characteristic scales (K_d, M_d vs. T_a, Y), and a larger reported residual for one generator does not by itself mean worse realization — it can equally reflect a larger requested acceleration under the same physical limits. This is a caveat about comparing reported magnitudes across generators, not a defect in r_k 's definition or in Theorem 1: the theorem is a claim about the optimization problem's structure being unchanged across generators, not about their residual magnitudes being numerically comparable, and the two should not be conflated.

Finally, the architectural novelty must be assessed against the full model predictive impedance/interaction-control literature surveyed in Section 2, not only the two closest precedents named there. The strongest eventual claim should rest on implemented generator modularity, formal residual guarantees, and matched experiments—not terminology alone.

10. Conclusion

This paper's central result is architectural, and Theorem 1 (Section 6) makes it precise: the realization layer's cost, constraints, and decision variable depend on which generator is plugged in through exactly one substitution, (C_θ, G_θ) , and nowhere else. One predictive QP, unchanged, therefore enforces impedance, admittance, and — on the Section 4.3/8.4 roadmap — energy-tank passive dynamics, human internal models, and learned models alike. The interaction model generates a desired acceleration; the QP realizes it as closely as the robot and safety constraints permit, and reports the shortfall as an explicit residual rather than absorbing it silently into a re-tuned impedance. In the planar simulation, this separation makes the safety intervention explicit and prevents large state violations that force clipping cannot anticipate, identically for both generators, without retuning the realization layer between them. Section 7.3 exercises the theorem directly rather than only across separate runs: one controller instance, one attribute reassigned online, impedance to admittance and back, with no discontinuity beyond the controller's own rate limit. Section 8 shows the same architecture — same cost structure, same realization layer, only the generator swapped — holds on a real manipulator model: on a torque-controlled FR3 in MuJoCo, predictive realization keeps a sustained push within millimeters of a workspace bound that reactive clipping overshoots by centimeters, with per-joint torque feasibility enforced at every predicted step rather than only the first. Getting there required correcting an unmodeled torque-projection path that produced a genuine closed-loop instability under sustained interaction — a manipulator-specific failure mode with no planar analogue, and a concrete illustration of why the point-mass result alone does not establish the manipulator claim. The next scientific step is not a better impedance tracker; it is implementing the remaining rows of Section 4's generator table — a genuinely nonlinear or learned generator realized by this same layer — together with disturbance and mass-mismatch sweeps, matched comparisons to existing predictive impedance and interaction controllers, and eventually human-participant and hardware validation.

References

1. A. Anand et al., “Deep Model Predictive Variable Impedance Control,” 2022. [arXiv:2209.09614](#).
2. K. Haninger et al., “Model Predictive Variable Impedance Control,” 2022. [arXiv:2208.07035](#).
3. S. Xue et al., “Model predictive variable impedance control towards safe robotic interaction in unknown disturbance-rich environments,” *Robotics and Autonomous Systems*, 2025. [Publisher page](#).
4. M. Sharifi, S. Behzadipour, and G. Vossoughi, “Nonlinear model reference adaptive impedance control for human–robot interactions,” *Control Engineering Practice*, 2014. [Publisher page](#).

5. M. Bednarczyk et al., “Model Predictive Impedance Control,” *IEEE International Conference on Robotics and Automation*, 2020. DOI: [10.1109/ICRA40945.2020.9196969](https://doi.org/10.1109/ICRA40945.2020.9196969).
6. T. Gold, A. Völz, and K. Graichen, “Model Predictive Interaction Control for Robotic Manipulation Tasks,” *IEEE Transactions on Robotics*, 2023. DOI: [10.1109/TRO.2022.3196607](https://doi.org/10.1109/TRO.2022.3196607).

Reproducibility

Run from `pHRI/imp_reference` :

```
# Architecture schematic (Figure 1)
python3 simulation/make_architecture_diagram.py

# Planar point-mass study (Section 7, Figure 2)
python3 simulation/run_experiments.py
python3 -m pytest simulation/test_interaction_dynamics_mpc.py simulation/test_benchmark_

# Online generator-switching demonstration (Section 7.3, Figure 3)
python3 simulation/run_generator_switching_experiment.py
python3 -m pytest simulation/test_generator_switching.py -q

# FR3/MuJoCo manipulator study (Section 8, Figure 4)
python3 simulation/run_fr3_experiments.py
python3 -m pytest simulation/test_fr3_interaction_dynamics_mpc.py simulation/test_fr3_benchmark_
```

Both driver scripts use a fixed deterministic scenario and record every numeric parameter and reported metric in `results/metrics.json` and `results/fr3_metrics.json`, respectively, but neither asserts anything itself. The `test_interaction_dynamics_mpc.py` / `test_fr3_interaction_dynamics_mpc.py` suites check the QP's mechanics on individual, hand-picked states; they do not run the 6-second scripted-force scenario. Tables 1 and 2 report numbers from. `test_benchmark_verification.py` / `test_fr3_benchmark_verification.py` close that gap: they run the actual benchmark scenario end-to-end and assert its headline claims directly — no torque-limit violation in any condition, no unhandled solver infeasibility, predictive realization within a controlled envelope of the workspace bound, and reactive clipping overshooting it by a checked margin — so a future change that silently breaks a claim in this paper fails a test rather than only showing up as a quieter number in a table nobody re-checks. Solve-time claims are bounded generously rather than asserted exactly, since they are wall-clock and were confirmed empirically to vary run-to-run (the position/torque numbers, by contrast, are exactly reproducible: re-running both benchmarks produced bit-for-bit identical values before this paper's figures and tables were finalized).

The FR3 study imports the shared MuJoCo/operational-space infrastructure from `pHRI/simulation` (`fr3_mujoco.py` , `fr3_impedance.py` , `so3_utils.py`) rather than duplicating it.